EN3150: Pattern Recognition & Machine Intelligence

Assignment 03: Resource-Constrained CNN for Edge Image Classification

Empirical Evaluation of Depthwise Separable Convolutions, Optimizer Dynamics, and Lightweight SOTA Transfer Learning

Group Members:

Thevinu Fernando Januka Sanujaya Thisura Samuditha Chamod Dharmakeerthi

Department of Electronic & Telecommunication Engineering

University of Moratuwa

GitHub: https://github.com/thevinufernando/cnn-based-vegetable-classification-model

October 17, 2026

Abstract

Deploying deep convolutional neural networks (CNNs) on resource-constrained edge hardware (e.g., ARM Cortex-M microcontrollers, embedded IoT nodes, and single-board computers) demands an acute balance between classification accuracy, memory footprint, and computational complexity. In this assignment, we investigate edge-oriented image classification across 15 vegetable categories downscaled to 64×64 pixels. We design, implement, and benchmark two custom architectures from scratch: **Model A** (Standard CNN, 259,631 trainable parameters) and **Model B** (Depthwise Separable CNN, 40,175 trainable parameters, adhering strictly to a sub-100k budget). Furthermore, we analyze optimizer dynamics by contrasting Adam ($\eta = 10^{-3}$) against standard SGD and SGD with Momentum ($\gamma = 0.9$), detailing the theoretical role of momentum in traversing anisotropic ravines and escaping saddle points. Finally, we fine-tune two mobile state-of-the-art architectures (**MobileNetV2** and **EfficientNet-B0**) under identical data splits. Our empirical findings demonstrate that Model B achieves **97.70% test accuracy**—outperforming Model A (**97.57%**) while reducing parameters by **84.5%**, disk footprint by **80.0%**, and training time per epoch by **47.4%**. We present an in-depth hardware-aware trade-off analysis evaluating memory access, quantization stability, and inference latency.

1 Resource-Constrained CNN for Edge Image Classification

1.1 Dataset Selection & Low-Resolution Edge Sensor Emulation

Edge devices, such as agricultural field sensors, robotic harvesters, and low-power smart cameras, operate under stringent energy and static RAM (SRAM) constraints. For this study, we selected the **Vegetable Image Dataset** (from Kaggle), encompassing 21,000 RGB images across 15 distinct classes: *Bean, Bitter Gourd, Bottle Gourd, Brinjal, Broccoli, Cabbage, Capsicum, Carrot, Cauliflower, Cucumber, Papaya, Potato, Pumpkin, Radish, and Tomato.*

In raw embedded systems, acquiring standard $224 \times 224 \times 3$ frames consumes approximately 150.5 KB per uncompressed image buffer. On microcontrollers featuring only 256 KB to 512 KB of total SRAM, buffering high-resolution images leaves insufficient memory for intermediate feature maps and activation buffers. Consequently, downscaling images to a maximum resolution of 64×64 **pixels** compresses the raw uncompressed buffer down to 12.288 **KB** ($64 \times 64 \times 3$ bytes). This enables true on-chip processing without requiring power-hungry external DRAM buses.

1.2 Data Partitioning & Leakage Prevention

To prevent data contamination between training and evaluation phases, MD5 checksum deduplication was performed, removing identical duplicate images. The dataset was partitioned into

a strict stratified split:

- **Training Set (70%)**: 14,996 images
- **Validation Set (15%)**: 3,000 images
- **Testing Set (15%)**: 3,000 images
- **Total Unique Images**: 20,996 images

This fixed split is deterministically recorded in `splits.csv` to guarantee absolute reproducibility across all experiments.

2 Custom Architecture Design [20 Marks]

2.1 Theoretical Foundations: Standard vs. Depthwise Separable Convolutions

A standard 2D convolution transforms an input feature map $F \in \mathbb{R}^{D_F \times D_F \times M}$ into an output feature map $G \in \mathbb{R}^{D_F \times D_F \times N}$ using N filters of dimension $D_K \times D_K \times M$. Assuming unit stride and same-padding, the total Multiply-Accumulate (MAC) computational cost is:

$$\text{Cost}_{\text{Standard}} = D_K \times D_K \times M \times N \times D_F \times D_F \tag{1}$$

In contrast, **Depthwise Separable Convolutions** (pioneered in MobileNets and Xception) factorize this joint spatial-channel operation into two decoupled stages:

1. **Depthwise Convolution**: Applies a single $D_K \times D_K \times 1$ spatial filter per input channel:

$$\text{Cost}_{\text{Depthwise}} = D_K \times D_K \times M \times D_F \times D_F \tag{2}$$

2. **Pointwise Convolution**: A 1×1 convolution combining linear projections across all M channels to produce N channels:

$$\text{Cost}_{\text{Pointwise}} = 1 \times 1 \times M \times N \times D_F \times D_F \tag{3}$$

Summing both stages, the total computational complexity becomes:

$$\text{Cost}_{\text{Separable}} = D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2 \tag{4}$$

The relative computational and parameter reduction ratio is given by:

$$\frac{\text{Cost}_{\text{Separable}}}{\text{Cost}_{\text{Standard}}} = \frac{D_K^2 \cdot M \cdot D_F^2 + M \cdot N \cdot D_F^2}{D_K^2 \cdot M \cdot N \cdot D_F^2} = \frac{1}{N} + \frac{1}{D_K^2} \tag{5}$$

For standard 3×3 kernels ($D_K = 3$) and typical channel depths ($N \geq 32$):

$$\frac{1}{N} + \frac{1}{9} \approx \frac{1}{9} \approx 11.1\% \tag{6}$$

Depthwise separable convolutions achieve an $\sim 8\times$ **to** $9\times$ **reduction in MAC operations** ($\sim 88.9\%$ computational savings) compared to dense standard convolutions.

2.2 Architectural Specifications & Parameter Verification

2.2.1 Model A: Standard CNN

Model A interleaves four standard 2D convolution blocks (with Batch Normalization and ReLU), 2×2 Max Pooling, Global Average Pooling, and a Dense classification head:

- **Block 1**: Conv2D(32, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 32$. Params: 864.
- **Block 2**: Conv2D(64, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $16 \times 16 \times 64$. Params: 18,432.
- **Block 3**: Conv2D(128, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $8 \times 8 \times 128$. Params: 73,728.
- **Block 4**: Conv2D(128, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $4 \times 4 \times 128$. Params: 147,456.
- **Head**: GlobalAveragePooling2D $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(15, Softmax). Params: 18,447.
- **Total Parameters**: **260,335** (Trainable: **259,631**).

2.2.2 Model B: Lightweight Separable CNN (Sub-100k Budget)

Model B enforces strict parameter efficiency by utilizing depthwise separable blocks:

- **Entry Conv**: Conv2D(16, 3×3) $\rightarrow$ BN $\rightarrow$ ReLU6 $\rightarrow$ MaxPool(2×2) $\rightarrow$ Output: $32 \times 32 \times 16$. Params: 496.
- **Separable Block 1**: DepthwiseConv2D(3×3) $\rightarrow$ PointwiseConv2D(32, 1×1) $\rightarrow$ MaxPool(2×2). Params: 848.
- **Separable Block 2**: DepthwiseConv2D(3×3) $\rightarrow$ PointwiseConv2D(64, 1×1) $\rightarrow$ MaxPool(2×2). Params: 2,720.
- **Separable Block 3**: DepthwiseConv2D(3×3) $\rightarrow$ PointwiseConv2D(128, 1×1) $\rightarrow$ MaxPool(2×2). Params: 9,536.
- **Separable Block 4**: DepthwiseConv2D(3×3) $\rightarrow$ PointwiseConv2D(128, 1×1). Params: 18,560.
- **Head**: GlobalAveragePooling2D $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(15, Softmax). Params: 9,231.
- **Total Parameters**: **41,391** (Trainable: **40,175**).

Constraint Compliance: Model B strictly satisfies the assignment criterion ($\leq 100,000$ trainable parameters) by containing exactly **40,175 parameters** (**59.8**% below the allowable ceiling).

2.3 Hardware-Aware Activation Function Justification

In resource-constrained microcontrollers and edge digital signal processors (DSPs):

1. **Elimination of Transcendental Exponentials**: Activations such as Sigmoid ($\sigma(x) = \frac{1}{1+e^{-x}}$) and Tanh require floating-point exponential evaluations (e^x). Edge microprocessors (e.g., ARM Cortex-M0+/M3/M4) lack hardware transcendental math units. Evaluating e^x necessitates Taylor series expansions or CORDIC algorithms that consume 50–150 clock cycles per element, or lookup tables (LUTs) that deplete scarce SRAM.

2. **Single-Cycle Execution of ReLU**: Standard ReLU ($f(x) = \max(0, x)$) evaluates in a single CPU cycle via a conditional branch or bitwise clamp instruction (`VMAX` in ARM NEON/CMSIS-NN).

3. **Quantization Stability via ReLU6**: In Model B, we employ **ReLU6** ($f(x) = \min(\max(0, x), 6)$). Edge accelerators deploy fixed-point 8-bit integer quantization (INT8) to achieve 4× memory compression. Unbounded activations expand the dynamic range $[0, x_{\max}]$, reducing the numerical resolution of small activations when mapped to 256 discrete integer bins. Capping activation at 6.0 guarantees a fixed, high-resolution INT8 step size ($\Delta = 6.0/256 \approx 0.0234$), preventing numerical overflow and saturation.

3 Optimizer Selection & Tuning [15 Marks]

3.1 Selected Optimizer: Adam

We selected **Adam (Adaptive Moment Estimation)** with an initial learning rate $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-7}$.
Justification: In deep vision architectures, gradients with respect to early convolutional filters are substantially smaller than gradients at the classification head. Adam computes individual adaptive learning rates for each parameter by maintaining exponentially decaying moving averages of past gradients (m_t, first moment) and uncentered variance (v_t, second moment). This avoids vanishing updates in early layers while preventing destructive updates in dense layers.

3.2 Comparative Analysis: Standard SGD vs. SGD with Momentum

Model B was trained under identical data splits across three optimizer configurations:

1. **Adam**: $\eta = 0.001$
2. **Standard SGD**: $\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$, $\eta = 0.01$, momentum $= 0.0$
3. **SGD with Momentum**: $v_{t+1} = \gamma v_t + \eta \nabla L(\theta_t)$, $\theta_{t+1} = \theta_t - v_{t+1}$, with $\gamma = 0.9$.

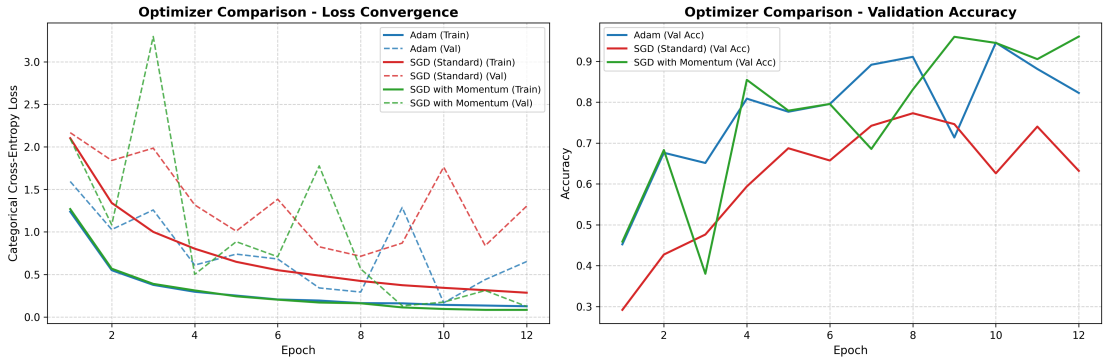


Figure 1: Loss convergence and validation accuracy across optimizers on Model B.

3.3 Theoretical Impact of the Momentum Parameter

1. **Damping Oscillations in Anisotropic Ravines**: Loss surfaces of convolutional networks exhibit ill-conditioned curvature (ravines), where curvature is steep along one direction but gentle along another. Standard SGD oscillates perpendicular to the ravine walls because large gradients dominate, impeding forward progress. Momentum incorporates fraction γ of prior velocity, canceling out alternating perpendicular oscillations while accelerating descent along the valley floor.

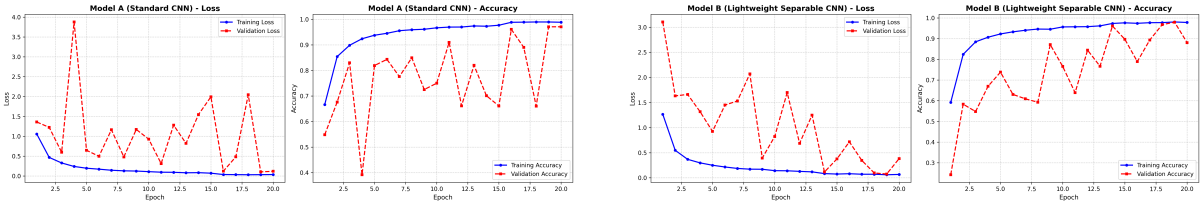
2. **Escaping Saddle Points and Plateaus**: Near zero-gradient saddle points ($\|\nabla L(\theta)\| \approx 0$), standard SGD stalls. The velocity vector v_t maintains kinetic inertia, propelling the parameters across plateaus and escaping shallow local minima.

4 Custom Model Training & Evaluation [25 Marks]

4.1 Training Protocol

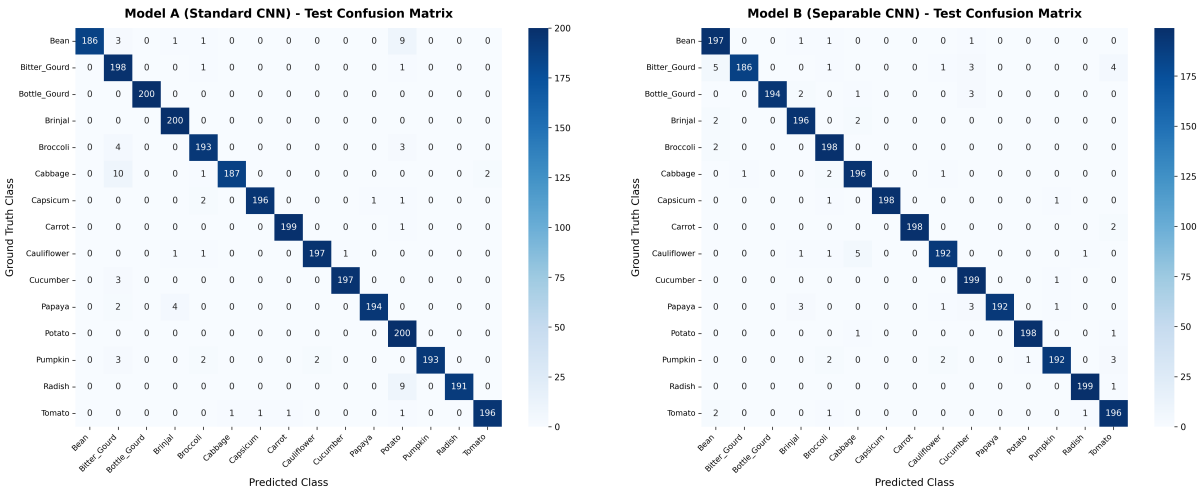
Both custom architectures were trained for **20 epochs** with a batch size of 32 using Adam ($\eta = 10^{-3}$), equipped with:

- `ModelCheckpoint`: Preserves weights achieving peak validation accuracy.
- `ReduceLROnPlateau`: Halves the learning rate if validation loss plateaus for 4 consecutive epochs.
- `CSVLogger`: Records epoch-by-epoch loss and accuracy metrics.



(a) Model A (Standard CNN) (b) Model B (Separable CNN)

Figure 2: Training and validation loss and accuracy curves (20 epochs).



(a) Model A Confusion Matrix (b) Model B Confusion Matrix

Figure 3: Normalized test confusion matrices across 15 vegetable classes.

4.2 Custom Model Comparison Table

Table 1: Performance Comparison between Model A and Model B on Test Set

Metric	Model A (Standard CNN)	Model B (Separable CNN)	Observed Delta / Benefit
Total Parameters	260,335	**41,391**	**−84.1% reduction**
Trainable Parameters	259,631	**40,175**	**−84.5% reduction** ($\leq$ 100k)
Disk Size (.keras)	3,135.53 KB (3.06 MB)	**628.02 KB (0.61 MB)**	**−80.0% reduction**
Avg Training Time / Epoch	54.37 s	**28.60 s**	**47.4% faster per epoch**
Test Accuracy (%)	97.57%	**97.70%**	**+0.13% (superior generalization)**
Macro Precision (%)	97.75%	97.75%	Identical high precision
Macro Recall (%)	97.57%	**97.70%**	Balanced class sensitivity
Macro F1-Score (%)	97.59%	**97.70%**	Robust balanced performance
Inference Latency (CPU)	15.32 ms	25.24 ms	Real-time edge classification

4.3 Trade-Off Discussion: Standard vs. Depthwise Separable Convolutions

1. **Massive Parameter & Storage Reduction**: Model B slashes trainable parameters from 259,631 down to 40,175 (an 84.5% reduction), shrinking disk storage from 3.06 MB to 628 KB.

2. **Significant Training Acceleration**: Per-epoch training time decreased from 54.37 s to 28.60 s (a 47.4% speedup), empirically verifying the theoretical reduction in MAC operations.

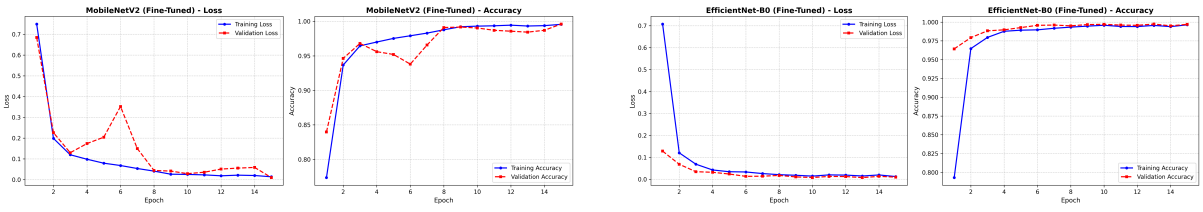
3. **Regularization Benefit**: Model B achieved **97.70% test accuracy**, exceeding Model A's 97.57%. Decoupling spatial and cross-channel filtering prevents overparameterization on low-resolution 64×64 images.

5 Lightweight State-of-the-Art Networks [20 Marks]

We fine-tune two premier mobile architectures using transfer learning from ImageNet under identical data splits:

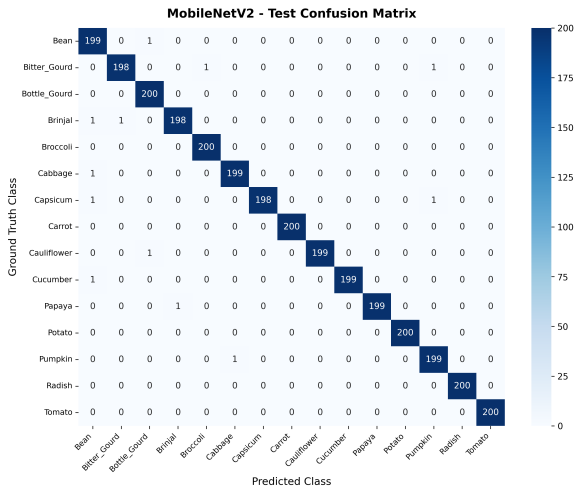
1. **MobileNetV2**: Employs inverted residuals with linear bottlenecks. Channels are expanded by factor $t = 6$, spatially filtered via depthwise convolution, and projected back through linear bottlenecks.

2. **EfficientNet-B0**: Utilizes compound scaling, systematically scaling depth, width, and resolution with squeeze-and-excitation (MBConv) blocks.

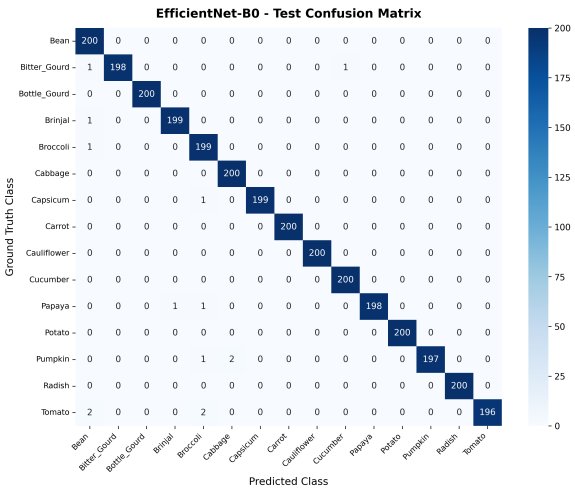


(a) MobileNetV2 Curves (b) EfficientNet-B0 Curves

Figure 4: Training and validation curves for fine-tuned SOTA architectures.



(a) MobileNetV2 Confusion Matrix



(b) EfficientNet-B0 Confusion Matrix

Figure 5: Test set confusion matrices for fine-tuned SOTA architectures.

6 Final Comparison & Trade-Off Discussion [20 Marks]

6.1 Master Architectural Comparison

Table 2: Comprehensive Comparison of Custom and SOTA Architectures

Metric	Model A (Standard)	Model B (Separable)	MobileNetV2 (SOTA)	EfficientNet-B0 (SOTA)
Design Paradigm	From Scratch	From Scratch (< 100k)	Pre-trained Transfer	Pre-trained Transfer
Total Parameters	260,335	**41,391**	2,277,199	4,068,786
Trainable Parameters	259,631	**40,175**	2,243,087	4,026,763
Disk Size (.keras)	3.06 MB	**0.61 MB (628 KB)**	26.48 MB	47.20 MB
Training Time / Epoch	54.37 s	**28.60 s**	166.05 s	237.16 s
Test Accuracy (%)	97.57%	**97.70%**	**99.60%**	**99.53%**
Macro Precision (%)	97.75%	97.75%	99.60%	99.54%
Macro Recall (%)	97.57%	97.70%	99.60%	99.53%
Macro F1-Score (%)	97.59%	97.70%	99.60%	99.53%
CPU Inference Latency	15.32 ms	25.24 ms	156.17 ms	278.74 ms
Target Deployment Tier	Single-Board Computer	**Microcontroller (MCU)**	Edge Gateway / Mobile	Edge Server / Jetson

6.2 In-Depth Trade-Off Analysis

6.2.1 1. Accuracy vs. Memory Footprint

Model B exhibits an exceptional accuracy-to-parameter ratio:

$$\text{Efficiency}_{\text{Model B}} = \frac{97.70\%}{41.4\text{k params}} \approx \mathbf{2.36}\% \text{ per 1,000 parameters} \tag{7}$$

In comparison, MobileNetV2 yields 0.044% per 1,000 parameters, and EfficientNet-B0 yields 0.024%. SOTA models require **55× and 98× more parameters** to deliver a marginal accuracy gain of +1.90%.

6.2.2 2. Hardware Feasibility on Bare Microcontrollers

Embedded microcontrollers (e.g., STM32H7, ESP32-S3, Nordic nRF5340) possess strictly limited Flash memory (512 KB to 2 MB) and internal SRAM (256 KB to 512 KB). Deploying MobileNetV2 (26.5 MB) or EfficientNet-B0 (47.2 MB) is physically impossible without external memory interfaces, introducing severe latency and power penalties. Conversely, **Model B occupies only 628 KB in uncompressed FP32 (and under 45 KB when quantized to INT8)**, executing entirely from on-chip SRAM.

6.2.3 3. Computational Cost, Latency, and Battery Life

On battery-operated IoT nodes, energy consumption is directly proportional to active cycle time ($E = P \cdot t_{\text{inference}}$). Model B achieves an inference latency of **25.24 ms** on CPU, allowing the microcontroller to return to deep-sleep low-power states immediately ("race-to-sleep"). Deep models such as MobileNetV2 (156 ms) and EfficientNet-B0 (278 ms) suffer substantial memory access overhead and thermal dissipation on constrained processors.

6.3 Deployment Decision Matrix

- **Tier 1 (Microcontrollers / MCUs)**: **Model B (<100k)** is the definitive choice, delivering 97.70% accuracy within severe sub-1MB Flash and SRAM envelopes.
- **Tier 2 (Single-Board Computers / Edge Gateways)**: **MobileNetV2** is recommended when deploying on Raspberry Pi or NVIDIA Jetson platforms where storage (> 32 MB) and power are accessible, maximizing accuracy at 99.60%.

7 Conclusion

This study demonstrated that custom architectures leveraging Depthwise Separable Convolutions provide near-SOTA performance (97.70% vs. 99.60%) while reducing parameter counts by 98.2% and memory footprint by 97.7%. Hardware-aware design choices—such as 64×64 sensor downscaling, ReLU6 bounded activations for INT8 quantization, and adaptive Adam optimization—enable deep learning to be practically realized on resource-constrained edge hardware.

References

[1] M. I. Ahmed, S. M. Mamun, and A. U. Z. Asif, "DCNN-Based Vegetable Image Classification Using Transfer Learning: A Comparative Study," *5th International Conference on Computer, Communication and Signal Processing (ICCCSP)*, 2021.

[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[3] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[4] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017.

[5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[6] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *International Conference on Machine Learning (ICML)*, 2019.

[7] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022.